`tl_jones_ff.py`

Code to compute Jones representation trace polynomials $\mathrm{tr}(\rho_\lambda(\beta))$, often more efficiently than symbolic matrix multiplication, using finite-field evaluations.

Implementation

We compute the trace polynomial $\mathrm{tr}(\rho_\lambda(\beta)) \in \mathbb{Z}[v, v^{-1}]$ as follows:

1. Write $\beta = \sigma_{i_1}^{\varepsilon_1} \cdots \sigma_{i_L}^{\varepsilon_L}$ with $\varepsilon_j \in \{\pm 1\}$ and choose a large prime p and a primitive root of unity ω in $\mathbb{F}_p^\times$.
2. Evaluate the Jones representation matrices of each braid generator at $v = \omega^0, \omega^1, \ldots, \omega^{2L}$,
3. Multiply these evaluated generator matrices to form evaluated braid matrices,
4. Take traces of these evaluated braid matrices to form evaluated traces in $\mathbb{F}_p$,
5. Use inverse DFT on these evaluated traces to form the trace polynomial's reduction mod p,
6. Lift the trace polynomial's reduction mod p to form the true trace polynomial.

We explain the choice of p and ω now. We know $\mathrm{tr}(\rho_\lambda(\beta))$ is of the form

$$t(v) := \sum_{m=-L}^{L} c_m v^m.$$

So it suffices to compute the shifted polynomial

$$s(v) := v^L\, t(v) = \sum_{j=0}^{2L} a_j v^j, \qquad a_j = c_{j-L}.$$

These coefficients are determined by $2L+1$ evaluations. Suppose the input word is nonempty, so that $L > 0$. Define $C := d^L$, where $d := \dim(V_\lambda)$. Choose the smallest prime p such that

$$p > 2C \qquad \text{and} \qquad p \equiv 1 \bmod 2L+1.$$

Furthermore, choose a primitive $(2L+1)^{\text{th}}$ root of unity ω in $\mathbb{F}_p$.

The latter condition on p ensures the root ω exists. The former condition on p ensures we can determine each coefficient c_m, and therefore each shifted coefficient a_j, from its residue modulo p. This is because each coefficient lies in the interval $[-C, C]$, which has length $2C < p$.

Now suppose we have evaluated traces $t(\omega^k) \in \mathbb{F}_p$ for $k = 0, 1, \ldots, 2L$. Then $s(\omega^k) = \omega^{kL}\, t(\omega^k)$ and we have

$$\omega^{kL}\, t(\omega^k) = \sum_{i=0}^{2L} a_i \omega^{ki}.$$

The inverse discrete Fourier transform gives

$$a_j = \frac{1}{2L+1} \sum_{k=0}^{2L} \omega^{kL} t(\omega^k)\, \omega^{-kj} \quad \text{in } \mathbb{F}_p,$$

which recovers the shifted coefficients modulo p. Each residue is then lifted to its symmetric integer representative in the interval $[-p/2, p/2]$. Since the true coefficients lie in $[-C, C]$ and $p > 2C$, this lift gives the exact integer coefficient. Finally, the code unshifts by using $c_{j-L} = a_j$, so

$$t(v) = v^{-L} s(v) = \sum_{j=0}^{2L} a_j v^{j-L}.$$

Overview

`tl_jones_ff.py`

1. `FFMatrix`
2. `FFEvaluationContext`
3. `ff_jones_polynomial`
4. `ff_jones_rep_trace`
5. `ff_jones_rep_braid_word`
6. `_ff_create_context`
7. `_inverse_dft`
8. `_ff_temperley_lieb_matrix`
9. `_identity_matrix`
10. `_matrix_multiply_mod`

The main public entry point is `ff_jones_polynomial`, which computes the same unnormalised Jones polynomial as `tl_jones.jones_polynomial`, but computes each representation trace using finite-field interpolation. The function `ff_jones_rep_trace` computes one such trace polynomial. The function `ff_jones_rep_braid_word` computes the evaluated braid matrices which feed into the trace computation. The remaining functions are internal helpers for building the finite-field context, applying inverse DFT, representing sparse Temperley–Lieb action data, and multiplying dense finite-field matrices.

Function Explanations

`FFMatrix` This is a type alias for a dense finite-field matrix, represented as a list of rows, where each row is a list of Python integers. The entries are always interpreted modulo the prime stored in the relevant `FFEvaluationContext`.

`FFEvaluationContext` Stores the shared finite-field data needed to evaluate and reconstruct one trace. It contains the strand number n, the index r for the module $V_{(n-r,r)}$, the validated braid word, the prime p, the interpolation order, the chosen root of unity, the degree bound, the coefficient bound, and the dimension of the link-state basis.

`ff_jones_polynomial` Computes the unnormalised Jones polynomial of the closure of a braid word. It validates the braid word, computes its writhe, then forms the same weighted trace sum as `tl_jones.jones_polynomial`:

$$(-v^2)^{-w(\beta)} \sum_{r=0}^{\lfloor n/2 \rfloor} [n-2r+1]_v \ \mathrm{tr}(\rho_{(n-r,r)}(\beta)).$$

The difference is that each trace $\mathrm{tr}(\rho_{(n-r,r)}(\beta))$ is computed by `ff_jones_rep_trace`.

`ff_jones_rep_trace` Computes the Laurent polynomial $\mathrm{tr}(\rho_{(n-r,r)}(\beta))$ by finite-field interpolation. It first builds an `FFEvaluationContext`, calls `ff_jones_rep_braid_word` to obtain the braid matrix evaluated at each interpolation point, takes the trace of each evaluated matrix modulo p, applies `_inverse_dft`, and finally lifts the residues to symmetric integer representatives to recover the exact Laurent polynomial.

`ff_jones_rep_braid_word` Evaluates the Jones representation of a whole braid word at every interpolation root $1, \omega, \ldots, \omega^{M-1}$, where M is the interpolation order. For each root it builds each generator matrix from cached Temperley–Lieb action data, multiplies the generator matrices in braid-word order, and returns the resulting list of dense finite-field braid matrices. This is the finite-field analogue of `tl_jones.jones_rep_braid_word`, except that it returns one evaluated matrix for each interpolation point rather than one symbolic matrix.

`_ff_create_context` Validates n, r, and the braid word, computes $d = \dim(V_{(n-r,r)})$, and chooses the prime and root of unity used for reconstruction. For the empty word it uses degree bound 0, interpolation order 1, and root of unity 1. For a nonempty word of length L, it uses degree bound L, interpolation order $2L+1$, coefficient bound d^L, and chooses a prime $p > 2d^L$ with $p \equiv 1 \pmod{2L+1}$, so that a primitive $(2L+1)$-st root of unity exists in $\mathbf{F}_p$.

`_inverse_dft` Recovers the shifted Laurent coefficients modulo p from the evaluated trace values. Given values $t(\omega^k)$, it first multiplies by ω^{kL} to shift the Laurent polynomial into an ordinary polynomial of degree at most $2L$, then applies the inverse discrete Fourier transform over $\mathbf{F}_p$. The returned list contains residues corresponding to the powers $v^{-L}, v^{-L+1}, \ldots, v^L$.

`_ff_temperley_lieb_matrix` Computes cached sparse column-action data for the Temperley–Lieb generator E_i on the link-state basis of $V_{(n-r,r)}$. For each basis vector it records the coefficient of the image and the row index of the resulting basis vector. This stores the same information as `tl_jones._temperley_lieb_matrix`, but avoids constructing a symbolic SymPy matrix.

`_identity_matrix` Builds the dense identity matrix of a given size as a list of lists of integers. It is used as the starting matrix when multiplying the evaluated generator matrices for a braid word.

`_matrix_multiply_mod` Multiplies two dense integer matrices modulo p. The implementation skips zero entries in the left and right factors, which is useful because many evaluated Temperley–Lieb generator matrices are sparse or nearly sparse.